

Improving Scientific Reasoning in Vision-Language Models using Code Sandboxes, Agentic Correction, Learned Verification, and RAG on mmJEE-Eval

Arpit Kumar (12340350), Saurav Gupta (12341940)

I. PROJECT DETAILS

Criterion followed. *Code available; we address a stated limitation of the base paper and propose an improvement.* The base paper, *mmJEE-Eval* [1], publicly releases its dataset and evaluation code, but it provides only an evaluation harness, not a solver. We reuse the released benchmark and its answer-extraction and scoring functions without changes, and we build a new inference-time reasoning framework on top of a frozen vision-language model (VLM). We do not train or fine-tune any VLM. The only component we train is a lightweight solution verifier.

Team composition and individual contributions:

- **Arpit Kumar (12340350)**, email: *arpitk@iitbhillai.ac.in*. Responsible for the Retrieval-Augmented Exemplar Prompting module (knowledge-base construction, retrieval, and exemplar injection).
- **Saurav Gupta (12341940)**, email: *sauravg@iitbhillai.ac.in*. Responsible for the Agentic Iterative Correction module (Solver/Critic/Corrector orchestration).
- **Both members (shared work)**: the Code-Sandbox Tool Use module and the Learned Solution Verifier (the core machine-learning component), including data generation, feature design, training, and evaluation.

II. PROBLEM STATEMENT

A. Context and Motivation

Vision-language models (VLMs) now score very highly on popular multimodal reasoning leaderboards, which makes it hard to tell genuine scientific reasoning apart from pattern matching [1]. The base paper *mmJEE-Eval* [1] addresses this by testing 17 models on 1,460 bilingual (English and Hindi) questions from India’s JEE Advanced examination (2019–2025), covering pre-college Physics, Chemistry, and Mathematics. The paper reports three findings that motivate our work:

- A large capability gap: on the held-out 2025 questions, frontier models reach 77–84% accuracy while open-source models stay at 37–45%.
- A contamination test suggests the difficulty comes from reasoning depth rather than memorization: frontier models drop 2–5% on the unseen 2025 set, while open models stay roughly stable.

- A metacognition weakness: models detect an error in their own reasoning in 21–73% of cases but fix it in only 1.1–5.2% of cases, well below the gains normally seen from repeated sampling.

These numbers are reported by the base paper; we take them as our starting point.

B. Why This Matters in the ML Domain

The base paper is diagnostic: it measures these weaknesses but does not try to reduce them. Two gaps are concrete and measurable: the distance between *detecting* an error and *fixing* it, and weak performance on numerical questions (which the base paper reports make up 44.7% of the benchmark) that need reliable calculation. Whether low-cost, training-free, test-time methods can improve VLM scientific reasoning is a useful and practical question for a student project, because it needs careful engineering and evaluation rather than large-scale model training.

C. Scope of Our Work

We propose a modular inference-time framework with four parts and plan to measure their combined effect on the base benchmark:

- **Code-Sandbox Tool Use**: move arithmetic and algebra into executed code, aimed at the numerical questions.
- **Agentic Iterative Correction**: a Solver/Critic/Corrector loop that turns single-pass correction into an iterative process.
- **Learned Solution Verifier**: a trained classifier that scores candidate solutions and selects the final answer (our core ML component).
- **Retrieval-Augmented Exemplar Prompting**: retrieve previously solved problems of the same method and add them as worked examples.

We plan to study a single open substrate model and, optionally, a hosted model, and to report results only on the contamination-clean 2025 held-out subset.

III. METHODOLOGY

Our framework is a pipeline of four modules that wrap a frozen VLM (Fig. 2). Nothing in the VLM is trained; the only trained component is the verifier. We plan to add each module one at a time so that its individual effect can be measured.

A. Code-Sandbox Tool Use

a) **Weakness addressed:** Numerical questions are the largest category and reward reliable calculation, where VLMs often make arithmetic errors.

b) **Approach:** Following Program-of-Thoughts prompting [2], the model reasons in natural language but writes each calculation as a Python/SymPy program; the program’s result is used to form the final answer.

c) **Implementation:** The code is run in a sandboxed subprocess with a time limit and restricted imports. The computed value is returned to the model, which then emits the final answer in the `\boxed{ }` format that the benchmark expects. If the code fails to parse or run, we fall back to plain chain-of-thought prompting, so this module should not score below the baseline.

B. Agentic Iterative Correction

a) **Weakness addressed:** The base paper reports that models detect errors far more often than they fix them, and it evaluates correction in a single pass.

b) **Approach:** Three roles, realized as the same or (preferably) different VLMs with distinct prompts, work together: a *Solver* produces a reasoning chain; a *Critic* tries to locate the first wrong step instead of giving a yes/no verdict; a *Corrector* re-derives the solution from that step. The loop repeats for a bounded number of rounds with an explicit stopping rule, in the spirit of self-refinement with feedback [3].

c) **Implementation:** We pass the located step and the earlier reasoning between roles as plain text. To reduce shared blind spots, we plan to use a cross-model Critic where resources allow, and to report same-model versus cross-model settings as a comparison.

C. Learned Solution Verifier (core ML component)

a) **Weakness addressed:** Iterative correction can also turn a correct answer into a wrong one, because a model’s own confidence is not a reliable selector.

b) **Approach:** We replace self-selection and majority voting with a trained classifier that predicts the probability that a candidate solution is correct, and use it to pick the final answer from several candidates (best-of- N). This is an inference-time verifier in the style of trained solution verifiers [4]; it uses no reinforcement learning.

c) **Data:** We run the Solver on the 2019–2024 questions (disjoint from the 2025 test set) and sample several solutions per question. Each solution is labeled correct or incorrect automatically using the benchmark’s own scoring function against the gold answer. Because each question yields several solutions, roughly 1,270 training questions can produce on the order of 10^4 labeled solutions, giving a supervised binary-classification dataset at low annotation cost (Fig. 1).

d) **Features and models:** Each solution is described by sentence embeddings and simple interpretable features (length, number of steps, agreement with sibling samples [5], whether the code ran, and numeric plausibility). We plan to train and compare a few classifiers, such as logistic regression and gradient-boosted trees (XGBoost) [6], and to report verifier quality (AUC/F1) together with downstream accuracy.

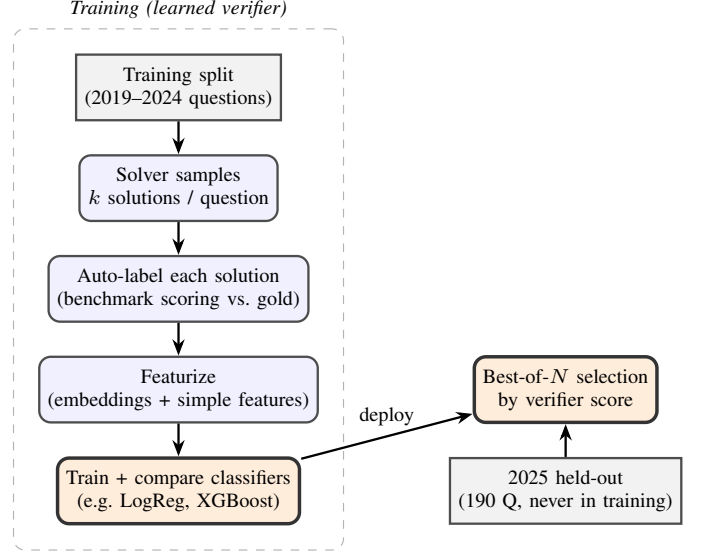


Fig. 1. Learned solution verifier. Labels are produced automatically from the gold answers. The classifier is trained only on 2019–2024 solutions and used on the disjoint 2025 held-out set, keeping train and test separate.

e) **Train/test separation:** Splits are made by year: we train on 2019–2024 and evaluate on 2025. Splitting by question ensures that solutions from the same question never appear on both sides. Correct solutions are the minority class, so we generate verifier training solutions using the same substrate model used for evaluation and address class imbalance using class weighting.

D. Retrieval-Augmented Exemplar Prompting

a) **Weakness addressed:** Weaker models often fail at choosing a solution method rather than at recalling formulas.

b) **Approach:** Instead of retrieving formulas (which models usually know), we retrieve previously solved problems that use the same method and add them as worked examples, following retrieval-augmented generation [7].

c) **Implementation:** The question image is first concept-tagged by the VLM to get a short text query (this avoids relying on raw OCR). We embed the tag, retrieve the top 2–3 examples from a knowledge base built only from 2019–2024 solved problems, and present them as guidance on method, not as answers to copy. We use a similarity threshold so that nothing is retrieved on a weak match. The knowledge base excludes and is de-duplicated against the 2025 test set to prevent leakage, and the learned verifier acts as a safety net if a retrieved example is misleading.

E. How Our Approach Differs

The base paper contributes a benchmark and a diagnosis, and it evaluates off-the-shelf models with single-pass prompting. Our project proposes a method: a combined, verifier-guided, tool-augmented multi-agent solver. For a course project, the contribution is in combining these components, adding a *trained* verifier in place of heuristic self-selection, and evaluating each part carefully on a hard benchmark.

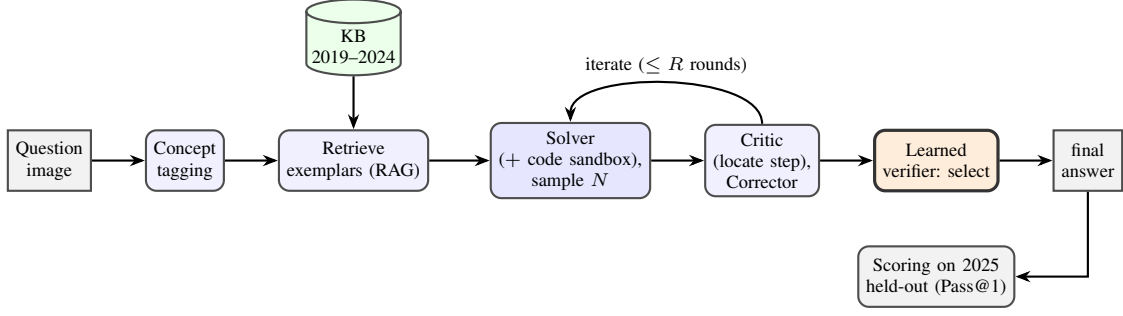


Fig. 2. Proposed inference-time pipeline. A frozen VLM is wrapped by four modules; only the verifier (orange) is trained. Retrieval and the knowledge base use only 2019–2024 data, and all reported accuracy is measured on the disjoint 2025 held-out subset using the base paper’s own scoring functions.

IV. RESOURCE AND DATASET AVAILABILITY

A. Dataset

We use the base benchmark directly.

- **Name and source:** mmJEE-Eval [1]. Project page: <https://mmjee-eval.github.io>. Code: <https://github.com/ArkaMukherjee0/mmJEE-Eval>. Data: Hugging Face ArkaMukherjee/mmJEE-Eval (MIT license).
- **Size and structure:** 1,460 questions from JEE Advanced 2019–2025 in English and Hindi, covering Physics, Chemistry, and Mathematics. Reported question types: Numerical 652 (44.7%), MCQ-Multiple 412 (28.2%), and MCQ-Single 396 (27.1%); the base paper reports that 442 questions (30.3%) require reading a diagram. Each question (stem, options, figure) is stored as an image with metadata fields (subject, type, year, paper, language, gold answer). The 2025 held-out set ($N=190$) comes from an exam held after the evaluated models’ cutoffs and serves as the contamination-clean test set.
- **Preprocessing and splits:** we load the data with the `datasets` library and split by year (2019–2024 for training/development, 2025 for evaluation). For the verifier we additionally generate solution samples and automatic labels (Section III-C). We do not use image augmentation; images are passed to the model directly, with a concept-tagging step used only for retrieval.
- **Suitability:** it is exactly the benchmark whose limitation we target, it is public and MIT-licensed, it provides gold answers that give free verifier labels, and it offers a contamination-clean split, which fits a training-free improvement study.

B. Data We Prepare Ourselves

No new human-annotated data is created. The verifier training data and the retrieval knowledge base are built only from the 2019–2024 portion (optionally with public JEEBench solved problems), keeping the 2025 test set untouched.

C. Resources Needed

- **GPU:** one GPU with about 16–24 GB memory (a lab workstation or a free Colab/Kaggle GPU). This is used to run an open substrate VLM under 4-bit or 8-bit

quantization and to produce solution samples for verifier training.

- **CPU/small GPU:** used to train the verifier (logistic regression / XGBoost) and to build the retrieval index; these are light and can run on a laptop.
- **Storage:** a few GB for the dataset, cached model generations, and the retrieval index.
- **Software:** Python, PyTorch, Hugging Face transformers and datasets, sentence-transformers, scikit-learn/xgboost, sympy, and a vector-search library (faiss).
- **Optional API:** a hosted VLM as a stronger substrate and for the cross-model Critic, used sparingly on the 190-item subset to keep cost low.

V. LIMITATIONS FOUND

- **Limitation we address:** the base paper states that its error-detection and error-correction study uses a single pass, while human solvers correct their work iteratively; it also reports that models fix only 1.1–5.2% of the errors they detect. Our Agentic Iterative Correction and Code-Sandbox modules target this gap, and the Learned Verifier is meant to keep iterative correction from making answers worse.
- **What differentiates our work:** it is a VLM-training-free, verifier-guided method (not a new benchmark), and its learned component is a small, practical ML model trained on automatically labeled data with a strict year-based train/test split.
- **Improvement we aim to study:** we plan to measure whether the framework improves Pass@1 over single-pass prompting on the 2025 held-out set, with the largest gains expected on the numerical questions and on harder items. We do not claim any result in advance; all numbers will come from our own experiments.
- **Fair comparison:** because multi-agent and best-of- N methods use more model calls, we plan to compare against self-consistency (majority voting) at a matched compute budget [5], so that any gain is not simply due to more computation.

VI. EXPECTED OUTCOMES

Deliverables.

TABLE I

PLANNED ADDITIVE EVALUATION ON THE 2025 HELD-OUT SUBSET. CELLS ARE TO BE FILLED AFTER EXPERIMENTS; “↑” MARKS THE EFFECT EACH MODULE IS EXPECTED TO HAVE. BASELINES INCLUDE SINGLE-PASS PROMPTING AND SELF-CONSISTENCY AT MATCHED COMPUTE.

Configuration	Numerical Acc.	Overall Pass@1
Baseline (single-pass CoT)	—	—
Self-consistency (matched)	—	—
+ Code sandbox	↑	↑
+ Agentic correction	—	↑
+ Learned verifier	—	↑
+ Retrieval (RAG)	—	↑
Full framework	—	—

- A reproducible inference-time framework (the four modules) around a frozen VLM, reusing the base paper’s scoring functions.
- A trained solution verifier with a comparison of a few classifiers, reported by AUC/F1.
- An additive evaluation on the 2025 held-out set (Table I) with a compute-matched self-consistency baseline, plus a breakdown by question type.
- Code, the generated solution/label data, the retrieval knowledge base, and a written project report.

Targeted metrics. We aim for a measurable Pass@1 improvement over single-pass prompting on the chosen substrate model, and for a verifier that selects answers better than majority voting. We state these as goals, not results; final numbers depend on the substrate model and compute budget, and nothing is claimed until measured.

REFERENCES

- [1] A. Mukherjee and S. Ghosh, “mmjee-eval: A bilingual multimodal benchmark for evaluating scientific reasoning in vision-language models,” in *Findings of the Association for Computational Linguistics: IJCNLP-AACL 2025*. Association for Computational Linguistics, 2025, pp. 2268–2290.
- [2] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *Transactions on Machine Learning Research*, 2023.
- [3] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [4] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [5] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

APPENDIX

Tools, platforms, and prompts. We use Python with Hugging Face `transformers` and `datasets`,

`sentence-transformers`, `scikit-learn/xgboost`, `sympy`, and `faiss`; open VLMs are run locally (single GPU, quantized) and, optionally, a hosted VLM via API. The base benchmark’s released answer-extraction and scoring functions are reused without changes. The module prompts (Solver, Critic step-location, Corrector, concept-tagging, and verifier featurization) will be documented in the final report and code repository. Large-language-model assistants were used to help draft and organize this proposal; all technical claims and dataset statistics cited here were checked against the original paper [1].